

前端模块化—复习要点

1 前端独立开发与模块化的由来

1.1 前后端分离后的前端独立开发

- 前后端分离后，前端需要独立进行开发，拥有自己的开发工具和项目目录结构 [p.8]。
- 登录功能范例暴露的问题：
 - 必须手动引入所有 JS 文件，且引入顺序非常重要 [p.9]。
 - 全局变量污染严重，命名冲突风险高，函数全部暴露在全局作用域 [p.10]。
 - 代码繁琐、可读性差、代码质量低，不利于业务逻辑复杂的应用 [p.11]。
 - 一个页面可能多次异步请求服务器，服务器压力大 [p.11]。
 - 无规范的开发流程和构建标准，难以重用 [p.11]。

1.2 前端工程化与模块化的概念

- **工程化**：前端工程化属于软件工程的一种，包括模块化、组件化、规范化和自动化四个方面，解决可维护性、可复用性和可扩展性问题 [p.12]。
- **模块化**：将一个大工程 (大文件) 拆分成相互依赖的小工程 (小文件)，编译或运行时再统一拼装和加载。狭义的前端工程化有时特指模块化 [p.12]。
- 工程化和模块化的深入推动了大量技术和工具的诞生：ES6、Node.js、NPM、CommonJS、Webpack、Vite、Angular、React、Vue 等 [p.12]。

1.3 模块化的分类与内容

- **外部模块化 (External Modularization)**：引入第三方包或插件，通常由一个或多个 JS 文件组成 [p.13]。
- **内部模块化 (Internal Modularization)**：项目内部的分层或分类，通常内部一个模块由一个 JS 文件表示 [p.13]。
- 模块化的主要内容：
 - 外部模块的管理——Node.js 和 NPM [p.13]。
 - 内部模块的组织——CommonJS、ES6 Module 和构建工具 [p.13]。
 - 模块源码到目标代码的编译和转换——Babel 等 [p.13]。

2 Node.js 与 NPM

2.1 Node.js 简介

- Node.js 是一个跨平台的开源服务器端运行环境，可以在 Windows、Linux、Unix、macOS 上运行 [p.15]。
- Node.js 是后端 JavaScript 运行时（Back-end JavaScript Runtime Environment），运行在 V8 JavaScript 引擎上，使 JavaScript 可以脱离浏览器运行 [p.15–16]。
- 对等关系：Java 的 JDK $\leftrightarrow$ JVM，对应 Node.js $\leftrightarrow$ Chrome V8 [p.15]。
- Node.js 核心组成：
 - **V8 引擎 (V8 Engine)**：来自 Chrome 浏览器，让 JavaScript 脱离浏览器运行 [p.16]。
 - **事件驱动、非阻塞 I/O (Event-driven, Non-blocking I/O)**：高性能，适合处理大量并发请求 [p.16]。
 - **NPM (Node Package Manager)**：包管理器，管理第三方依赖 [p.16]。
 - **内置模块 (Built-in Modules)**：fs(文件系统)、http(HTTP 服务器)、path(路径处理) 等 [p.16]。

2.2 Node.js 对前端开发的意义

- **提供工程化基础**：本地运行环境、包管理器 (NPM)、脚本工具 (package.json)、模块系统 (CommonJS) [p.17]。
- **催生现代工具链**：构建工具 (Webpack、Vite)、框架 (Vue、React、Angular) [p.17]。
- **统一语言栈**：可实现前后端都用 JavaScript 开发 [p.17]。
- 建议使用版本管理工具（如 nvm）来管理 Node.js 版本 [p.19]。

2.3 NPM 包管理器 (Node Package Manager)

- NPM 内置于 Node.js 环境中，是 Node.js 的包管理工具 [p.20]。
- 使用 `npm init` 初始化项目，会生成 `package.json` 文件 [p.20]。
- NPM 两大功能：
 - **包仓库 (Registry)**：云端包存储仓库 [p.20]。
 - **CLI 命令行工具**：安装、卸载和管理包 [p.20]。
- 语义化版本 (Semantic Versioning)：采用主版本. 次版本. 修订号（如 2.1.x）的格式 [p.21]。

2.3.1 NPM 常用命令

- `npm install jquery`——安装最新版依赖到 `node_modules`，自动更新 `package.json` (dependencies) 并生成 `package-lock.json` [p.22]。
- `npm install jquery@2.1.x`——安装指定版本 [p.22]。
- `npm install --save-dev eslint` 或 `npm install -D eslint`——安装到 devDependencies，生产环境不包含 [p.22]。
- `npm install -g webpack`——全局安装命令行工具 [p.22]。

2.4 Yarn (Yet Another Resource Negotiator)

- Yarn 由 Facebook、Google 等公司联合推出，旨在弥补 NPM 的缺陷 [p.24]。
- NPM 主要缺点：安装速度慢（大项目尤其明显）、版本一致性问题 [p.24]。
- Yarn 的改进：
 - 并行安装、本地缓存，提高安装速度 [p.24]。
 - 提供 `yarn.lock` 文件锁定包版本 [p.24]。
 - 更简洁的输出信息、语义化的命令 [p.24]。

3 前端模块化体系（内部模块化）

3.1 原始阶段——直接引入 JS 文件

- 方式一：将不同的 JS 文件在 HTML 中逐一引入，每个文件代表一个模块 [p.27]。
- 方式二：将每个模块包裹在函数作用域（IIFE）中执行，通过执行匿名函数返回模块输出，避免污染全局环境 [p.28]。
- 主要问题：
 - 项目增大后，HTML 中包含大量 `<script>` 标签 [p.29]。
 - `<script>` 标签顺序不能准确反映模块间的依赖关系（树状或网状），容易因变量未加载而报错 [p.29]。
 - 代码逻辑关系难以理解，不便维护 [p.29]。
 - 同步加载模式容易导致页面卡死 [p.29]。
 - 仍可能存在全局变量污染导致命名冲突 [p.29]。

3.2 在线处理阶段——AMD 与 CMD

- 在线处理阶段的核心思路：提供 API 和语法声明模块及其依赖关系，浏览器下载 JS 文件后根据声明分析依赖，逐步加载（在线编译） [p.30]。
- **AMD (Asynchronous Module Definition)**：异步模块定义，代表库为 `require.js` [p.30]。
- **CMD (Common Module Definition)**：通用模块定义，代表库为 `sea.js` [p.30]。
- 主要问题：
 - 在线组织模块延长了页面加载时间，影响用户体验 [p.30]。
 - 加载过程中发出大量 HTTP 请求，降低页面性能 [p.30]。

3.3 预处理阶段——CommonJS 与 ES Module

- 目前主流的 CommonJS 和 ES6 标准都属于预处理阶段模块化方式 [p.33]。
- 核心思想：提供特殊语法，将组织模块的工作提前做好，通过构建工具（如 Webpack）进行代码合并，在部署上线前完成模块化处理，从而节约页面加载时间并减少 HTTP 请求数量 [p.33]。

3.4 CommonJS 模块规范

- 每个文件就是一个模块，有自己的作用域；文件中定义的变量、函数、类都是私有的，对其他文件不可见 [p.34]。
- 每个模块内部，`module` 代表当前模块，`module.exports` 是对外的接口；加载某个模块实际上是加载其 `module.exports` 属性 [p.34]。
- 使用 `require()` 方法加载模块 [p.34]。
- 核心特点：
 - 所有代码都运行在模块作用域，不会污染全局作用域 [p.37]。
 - 模块可多次加载，但只在第一次加载时运行一次，之后读取缓存结果；要重新运行需清除缓存 [p.37]。
 - 模块加载的顺序按照在代码中出现的顺序 [p.37]。

3.5 ES Module (ES6 模块化)

- ES6 之前 JavaScript 没有自己的模块规范，社区制定了 CommonJS；ECMAScript 标准委员会借鉴了 CommonJS 的思想并写入 ES6 语法，成为官方标准 [p.38]。
- 语法简洁优雅，使用 `import / export` 关键字 [p.38]。
- 所有现代浏览器都默认支持 ES Module 模块化方式 [p.38]。
- 与 CommonJS 的关键区别：ES Module 是静态导入（编译时确定依赖），支持 tree-shaking；CommonJS 是动态导入（运行时确定依赖）。

4 构建打包工具

4.1 为什么要使用构建打包工具？

- 前端模块化的目标是**开发模块化**，而不是运行模块化 [p.40]。
- 实现步骤：
 1. 通过 CommonJS 或 ES6 实现 JS 的模块化编写 [p.40]。
 2. 通过 Webpack、Vite 等构建打包工具对各模块进行编译，处理请求合并、依赖去重、体积优化、兼容性处理、导入 npm 包等 [p.40]。
- 前端其他资源（HTML、CSS、图片等）也需要构建打包工具进行模块化 [p.40]。
- 构建打包工具的三大角色：
 - **翻译官**：将 ES6/TypeScript 等转译成浏览器可运行的代码 [p.41]。
 - **优化师**：压缩、去重、合并、tree-shaking 等优化 [p.41]。
 - **搬运工**：处理资源路径、复制静态文件等 [p.41]。

4.2 Webpack

- Webpack 是一个前端资源加载和打包工具，根据模块的依赖关系进行静态分析，按照指定规则生成对应的静态资源 [p.42]。
- 可将多种静态资源（JS、CSS、HTML 等）转换成一个静态文件，减少页面请求 [p.42]。

4.2.1 Webpack 五大核心概念

- **入口 (Entry):**
 - 入口起点指示 Webpack 应使用哪个模块作为构建内部依赖图 (Dependency Graph) 的开始 [p.44]。
 - 通过 `webpack.config.js` 中的 `entry` 属性配置一个或多个入口起点 [p.44]。
- **输出 (Output):**
 - `output` 属性告诉 Webpack 在哪里输出所创建的 bundles, 以及如何命名这些文件 [p.45]。
 - 通过 `webpack.config.js` 中的 `output` 属性配置 [p.45]。
- **加载器 (Loader):**
 - Webpack 自身只理解 JavaScript, 非 JS 资源 (如 CSS、图片等) 需要 Loader 进行加载和转换 [p.46]。
 - 例如加载 CSS 需要 `css-loader` 和 `style-loader`, 使用前需通过 NPM 安装 [p.46]。
- **插件 (Plugin):**
 - 插件执行比 Loader 范围更广的任务, 包括打包优化、压缩、重新定义环境变量等 [p.47]。
 - 使用插件需要先 `require`, 再添加到 `plugins` 数组中 [p.47]。
 - 如 `HtmlWebpackPlugin` 用于给 HTML 文件进行打包和处理 [p.47]。
- **模式 (Mode):**
 - Webpack 有两种打包模式: 开发模式 (Development) 和生产模式 (Production) [p.48]。
 - 生产模式会对打包结果进行加密和压缩 [p.48]。

4.3 Vite

- Vite 是新一代前端构建工具, 相比 Webpack 具有更快的开发服务器启动和热更新速度 [p.54]。
- 利用浏览器原生 ES Module 支持, 开发时无需打包 (No-bundle), 生产构建基于 Rollup。
- 与 Webpack 同属预处理阶段的构建打包工具 [p.33, p.54]。

5 本章小结

- 前端独立开发和模块化源于前后端分离后对代码组织、可维护性和可复用性的需求 [p.55]。
- Node.js 提供了 JavaScript 的本地运行环境和 NPM 包管理器, 是前端工程化的基础 [p.55]。
- 前端模块化经历了原始阶段 → AMD/CMD 在线处理阶段 → CommonJS / ES Module 预处理阶段三个演进过程 [p.55]。
- 构建打包工具 (Webpack、Vite) 将开发时的模块化代码转化为生产环境可高效运行的静态资源 [p.55]。

——完——